

LABORATORIO N° 03

Desarrollo de Software



DOCENTE:
Bestard Aroche, Yunion

CURSO:

Desarrollo de Aplicaciones
Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Diseñar e implementar Models en Django**, definiendo entidades, campos y relaciones básicas de acuerdo con los requisitos de una problemática planteada.
2. **Aplicar Django ORM y SQLite para gestionar datos persistentes**, utilizando migraciones y operaciones de creación, consulta, actualización y eliminación de registros.
3. **Implementar y explicar el flujo CRUD completo en Django**, integrando URLs, Views, Models, Templates y operaciones ORM para resolver una problemática real.

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

En la semana anterior se trabajó la estructura básica de una aplicación Django utilizando el patrón MVT, formularios, Views, URLs y Templates, pero los datos se almacenaban de forma temporal en memoria. En este laboratorio se dará el siguiente paso: convertir esas estructuras en información persistente mediante Django Models, migraciones, Django ORM y SQLite, de modo que los registros se mantengan disponibles incluso después de reiniciar el servidor.

El laboratorio se desarrollará en dos partes. En la primera, los estudiantes retomarán la aplicación realizada en la Semana 2 y adaptarán su funcionamiento para reemplazar los datos estáticos por Models reales y operaciones ORM, implementando la consulta y el registro de información de manera persistente. En la segunda parte, deberán investigar una problemática real, identificar sus requisitos y diseñar una solución que incluya varias entidades, relaciones entre Models y operaciones completas de creación, consulta, actualización y eliminación.

El objetivo es que el estudiante comprenda cómo Django conecta el código Python con una base de datos mediante el ORM y pueda aplicar este flujo de forma integral:

Request

→ **URL**

→ **View**

→ **Model**

→ **Manager / QuerySet**

→ **Django ORM**

→ **SQLite**

→ **Template**

→ **Response**

Al finalizar el laboratorio, el estudiante deberá ser capaz de diseñar Models, generar y aplicar migraciones, utilizar SQLite como mecanismo de persistencia e implementar operaciones CRUD sin escribir sentencias SQL manualmente. Asimismo, deberá comprender la diferencia entre modificar datos mediante CRUD y modificar la estructura de la base de datos mediante migraciones.

Enunciados

PARTE 1 — Migración del trabajo de la Semana 2 a Django ORM y SQLite

Ejercicio 1 — Recuperar y analizar la aplicación anterior

Abra la aplicación desarrollada en el laboratorio de la Semana 2 e identifique la entidad principal, los datos almacenados en memoria, las Views, URLs, formularios y Templates utilizados. Explique brevemente qué sucede con los registros agregados cuando se reinicia el servidor.

En la aplicación de la Semana 2, la entidad principal era la gestión de pizzas, y sus datos se guardaban temporalmente en listas de diccionarios dentro de la memoria RAM que es en `models.py`, para su funcionamiento se utilizaban vistas basadas en funciones en `views.py`, enrutamiento de rutas en `urls.py`, formularios basados en `forms.Form` y plantillas HTML con herencia de Bootstrap. Debido a esta arquitectura sin base de datos, cada vez que el servidor se reinicia, todos los registros agregados se pierden de forma permanente; esto sucede porque la memoria RAM libera su espacio asignado al detener el proceso de Python, haciendo que la aplicación regrese a su estado inicial programado en el código fuente.

Ejercicio 2 — Convertir la entidad en un Django Model

Reemplace la estructura de datos estática utilizada en models.py por un Django Model real. Mantenga los campos definidos a partir de los requisitos del laboratorio anterior y utilice tipos de datos apropiados de Django.

```
src > order > models.py
1  from django.db import models
2  from pizza.models import Pizza
3
4
5  class Cliente(models.Model):
6      id = models.AutoField(primary_key=True)
7      nombre = models.CharField(max_length=100)
8      telefono = models.CharField(max_length=20)
9      direccion = models.CharField(max_length=255)
10
11     def __str__(self):
12         return self.nombre
13
14     class Meta:
15         verbose_name = 'Cliente'
16         verbose_name_plural = 'Clientes'
17
18
19     class Order(models.Model):
20         ESTADO_CHOICES = [
21             ('Pendiente', 'Pendiente'),
22             ('En preparación', 'En preparación'),
23             ('Entregado', 'Entregado'),
24             ('Cancelado', 'Cancelado'),
25         ]
26
27         id = models.AutoField(primary_key=True)
28         cliente = models.ForeignKey(Cliente, on_delete=models.CASCADE, related_name='ordenes')
29         pizza = models.ForeignKey(Pizza, on_delete=models.PROTECT, related_name='ordenes')
30         cantidad = models.IntegerField(default=1)
31         fecha_pedido = models.DateTimeField(auto_now_add=True)
32         estado = models.CharField(max_length=50, choices=ESTADO_CHOICES, default='Pendiente')
33
34         def __str__(self):
35             return f"Pedido {self.id} - {self.cliente.nombre}"
36
37         class Meta:
38             verbose_name = 'Pedido'
39             verbose_name_plural = 'Pedidos'
```

```
src > pizza > models.py > Pizza > Meta
1  from django.db import models
2
3
4  class Pizza(models.Model):
5      MASA_CHOICES = [
6          ('Masa delgada', 'Masa delgada'),
7          ('Masa tradicional', 'Masa tradicional'),
8          ('Masa integral', 'Masa integral'),
9          ('Masa artesanal', 'Masa artesanal'),
10     ]
11
12     id = models.AutoField(primary_key=True)
13     nombre = models.CharField(max_length=100)
14     tipo_masa = models.CharField(max_length=50, choices=MASA_CHOICES)
15     ingredientes = models.TextField()
16     disponible = models.BooleanField(default=True)
17
18     def __str__(self):
19         return self.nombre
20
21     class Meta:
22         verbose_name = 'Pizza'
23         verbose_name_plural = 'Pizzas'
```

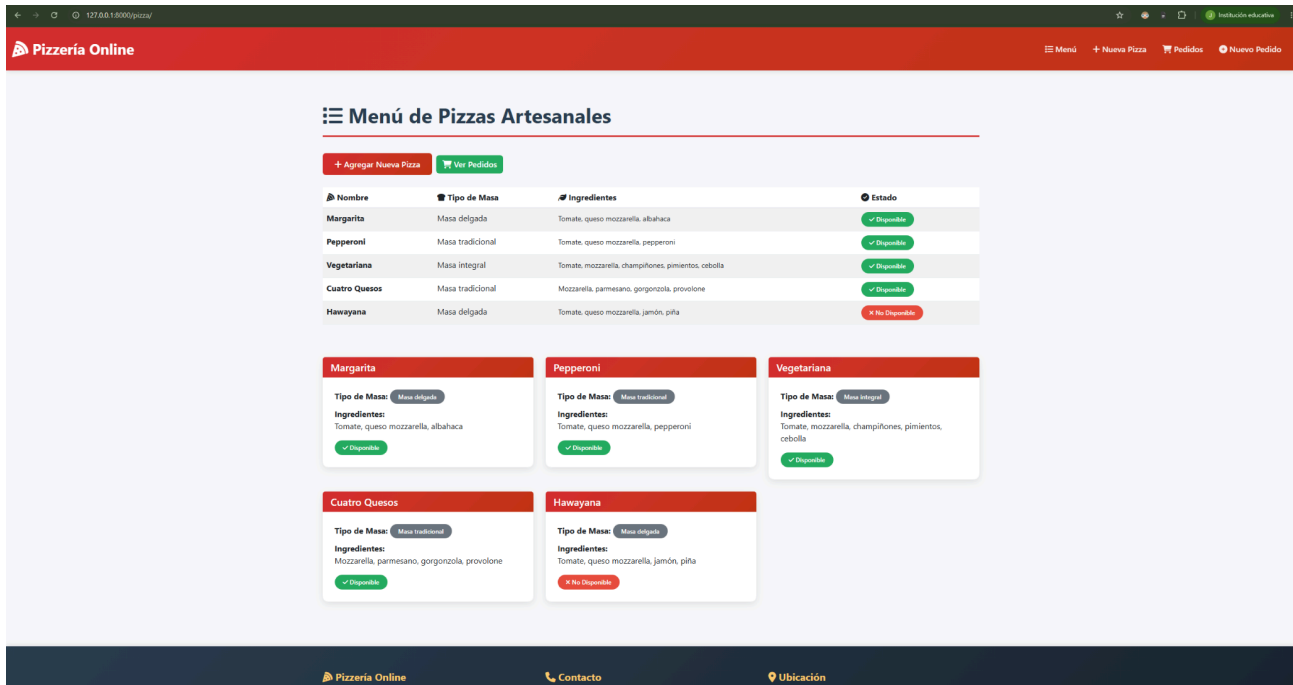
Ejercicio 3 — Crear la estructura persistente de datos

Genere y aplique las migraciones necesarias para crear la tabla correspondiente en SQLite. Documente mediante capturas el resultado de makemigrations, migrate y showmigrations.

```
(.venv) PS C:\Users\jfacu\Desktop\django_project> cd src
(.venv) PS C:\Users\jfacu\Desktop\django_project\src> python manage.py makemigrations
No changes detected
(.venv) PS C:\Users\jfacu\Desktop\django_project\src> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, core, order, pizza, sessions
Running migrations:
  No migrations to apply.
(.venv) PS C:\Users\jfacu\Desktop\django_project\src> python manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
core
[X] 0001_initial
[X] 0002_alter_item_id
[X] 0003_alter_item_created_at_alter_item_description_and_more
order
[X] 0001_initial
pizza
[X] 0001_initial
sessions
[X] 0001_initial
(.venv) PS C:\Users\jfacu\Desktop\django_project\src> 
```

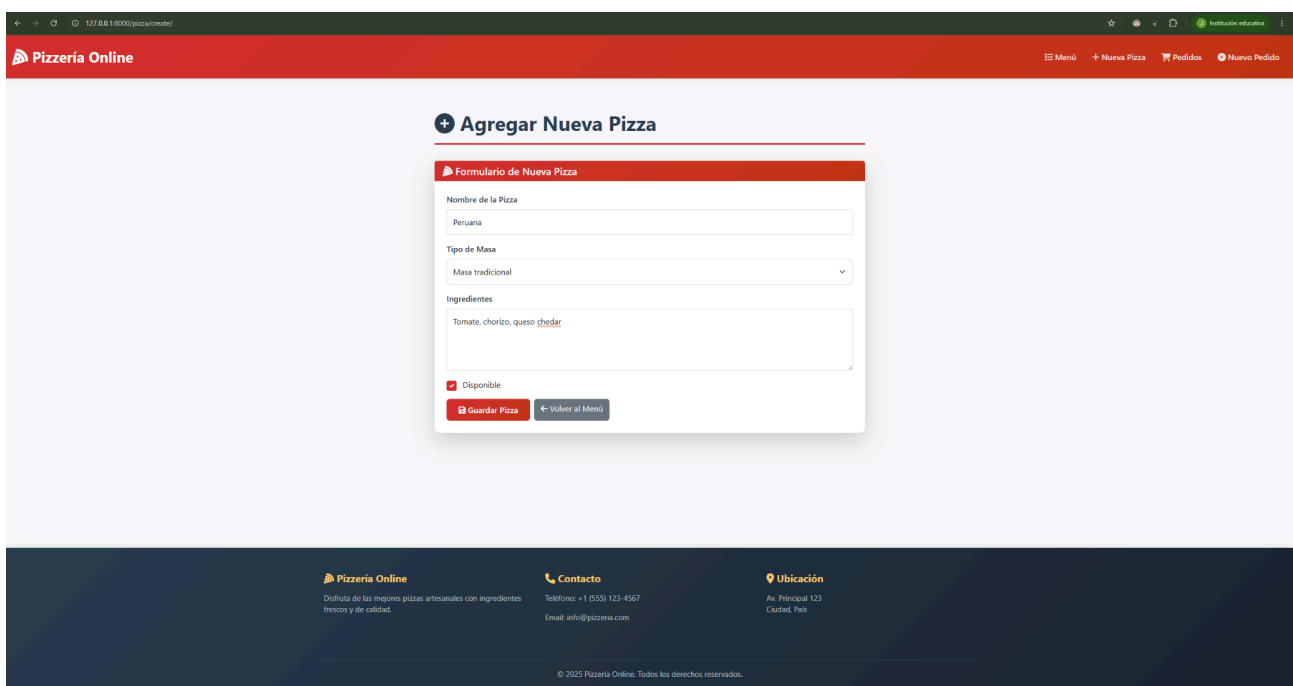
Ejercicio 4 — Implementar la consulta mediante ORM

Modifique la View encargada del listado para que deje de leer los datos desde una lista Python y los obtenga mediante Django ORM. Utilice un QuerySet y verifique que los registros almacenados en SQLite se muestran correctamente en el Template.



Ejercicio 5 — Implementar el registro mediante ORM

Adapte la funcionalidad de creación desarrollada en la Semana 2 para que los datos enviados desde el formulario sean almacenados de manera persistente utilizando Django ORM. Después del registro, redirija nuevamente al listado y compruebe que el nuevo dato permanece aunque se reinicie el servidor.





Nombre	Tipo de Masa	Ingredientes	Estado
Margarita	Masa delgada	Tomate, queso mozzarella, albahaca	✓ Disponible
Pepperoni	Masa tradicional	Tomate, queso mozzarella, pepperoni	✓ Disponible
Vegetariana	Masa integral	Tomate, mozzarella, champiñones, pimientos, cebolla	✓ Disponible
Cuatro Quesos	Masa tradicional	Mozzarella, parmesano, gorgonzola, provolone	✓ Disponible
Hawaiana	Masa delgada	Tomate, queso mozzarella, jamón, piña	✗ No Disponible
Peruana	Masa tradicional	Tomate, chorizo, queso cheddar	✓ Disponible

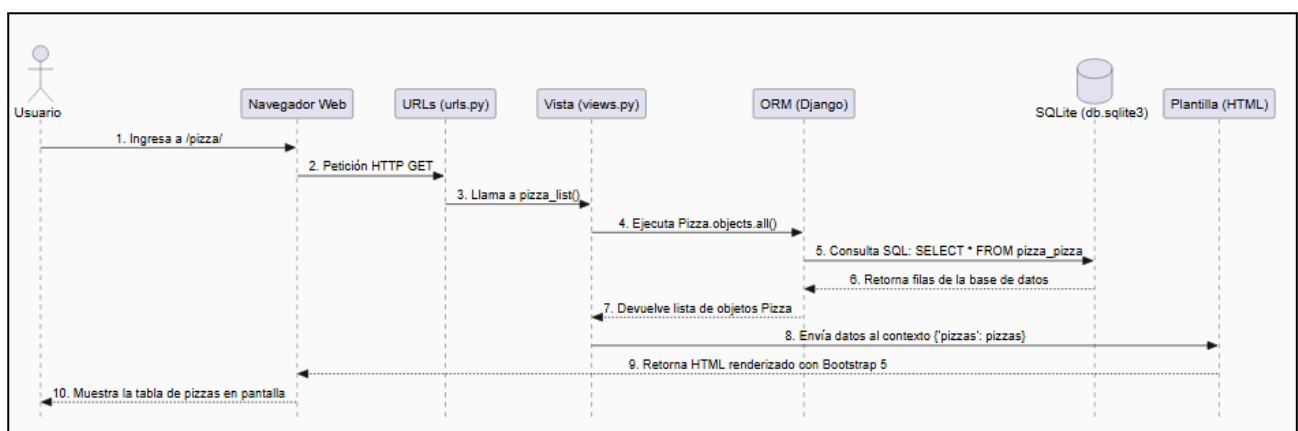
Ejercicio 6 — Analizar el flujo de persistencia

Documente el recorrido completo de una operación de consulta y una operación de creación:

Request

- URL
- View
- Model
- Manager / QuerySet
- Django ORM
- SQLite
- View
- Context
- Template
- Response

Identifique además qué operación SQL representa conceptualmente cada acción realizada mediante ORM.



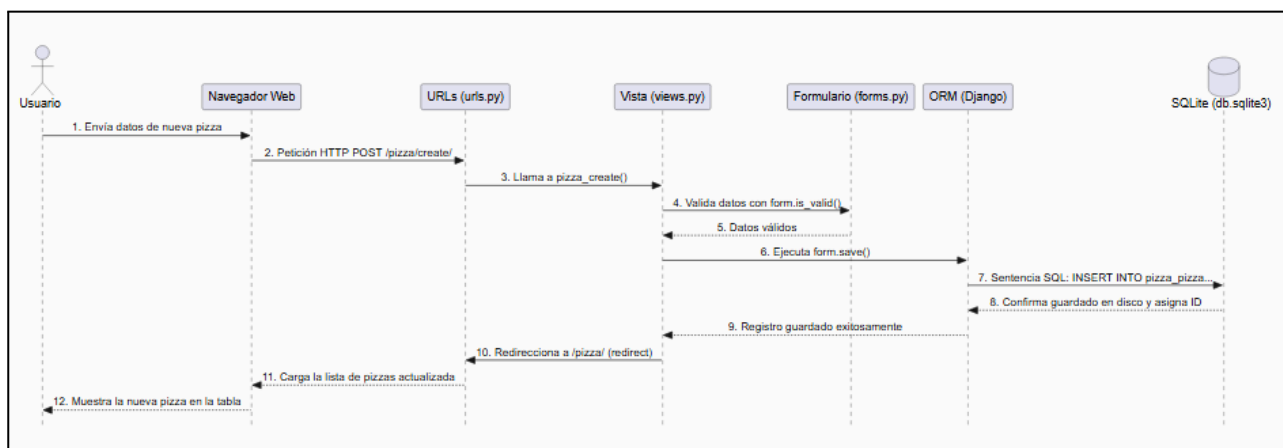


Tabla comparativa

Acción realizada mediante ORM	Operación SQL equivalente
Pizza.objects.all()	SELECT * FROM pizza_pizza;
Pizza.objects.get(id=1)	SELECT * FROM pizza_pizza WHERE id = 1 LIMIT 1;
form.save() (crear)	INSERT INTO pizza_pizza (...) VALUES (...);
pizza.save() (actualizar)	UPDATE pizza_pizza SET ... WHERE id = 1;
pizza.delete()	DELETE FROM pizza_pizza WHERE id = 1;

PARTE 2 — Investigación y desarrollo de una nueva solución persistente

Ejercicio 7 — Investigar una problemática real

Investigue una problemática real que pueda resolverse mediante una aplicación empresarial web. La problemática seleccionada debe manejar varios tipos de información y permitir justificar la existencia de múltiples entidades dentro de una base de datos. Describa el problema, los usuarios involucrados y el proceso que se desea mejorar.

Muchos complejos deportivos manejan sus reservas de canchas de forma manual (llamadas, WhatsApp o cuaderno físico), lo que genera errores como reservas duplicadas o pérdida de información. Los usuarios involucrados son el cliente, que busca reservar una cancha disponible, y el encargado, que administra canchas, horarios y pagos. Se busca una aplicación web que permita al cliente ver la disponibilidad por horario, reservar, y que el encargado gestione el registro de pagos de forma centralizada y persistente.

Ejercicio 8 — Definir los requisitos funcionales

Redacte entre 8 y 12 requisitos funcionales derivados de la problemática investigada. Los requisitos deben contemplar como mínimo operaciones para crear, listar, actualizar y eliminar información.

- El sistema debe permitir registrar nuevos clientes.
- El sistema debe permitir listar los clientes registrados.
- El sistema debe permitir registrar nuevas sedes.
- El sistema debe permitir registrar canchas asociadas a una sede.
- El sistema debe permitir listar las canchas disponibles por tipo de deporte.
- El sistema debe permitir crear horarios disponibles para cada cancha.
- El sistema debe permitir listar los horarios disponibles.
- El sistema debe permitir a un cliente crear una reserva sobre un horario disponible.
- El sistema debe permitir listar las reservas realizadas.
- El sistema debe permitir actualizar el estado de una reserva (pagada/no pagada).
- El sistema debe permitir eliminar una reserva.
- El sistema debe marcar un horario como no disponible una vez reservado.

Ejercicio 9 — Diseñar el modelo de datos

Identifique y diseñe como mínimo cinco entidades:

- tres entidades independientes o no relacionadas entre sí;

Entidad	Campos	Tipo de dato	Clave primaria	Clave foránea
Cliente	id, nombre, telefono, email	int, texto, texto, texto	id	—
Sede	id, nombre, direccion	int, texto, texto	id	—
Cancha	id, nombre, tipo, precio_hora	int, texto, texto, decimal	id	—

- dos entidades que mantengan una relación mediante ForeignKey.

Entidad	Campos	Tipo de dato	Clave primaria	Clave foránea
Horario	id, cancha, fecha, hora_inicio, hora_fin, disponible	int, FK, fecha, hora, hora, booleano	id	cancha → Cancha
Reserva	id, cliente, horario, fecha_reserva, pagado	int, FK, FK, fecha/hora, booleano	id	cliente → Cliente, horario → Horario

Para cada entidad defina:

- nombre;
- campos;
- tipos de datos;
- clave primaria;
- clave foránea cuando corresponda.

Justifique por qué cada entidad y cada relación son necesarias para resolver la problemática.

Se necesita Cliente para identificar quién reserva, Cancha para saber qué espacio se alquila, y Sede para ubicar el local. Horario depende de Cancha porque cada cancha tiene distintos bloques de disponibilidad, y Reserva depende de Cliente y Horario porque conecta quién reservó con qué horario específico, permitiendo controlar la disponibilidad real y evitar reservas duplicadas.

Ejercicio 10 — Representar las relaciones

Elabore un diagrama sencillo del modelo de datos donde se identifiquen claramente las cinco entidades y la relación entre las dos entidades vinculadas.

Ejemplo conceptual:

Entidad A

Entidad B

Entidad C

Entidad D

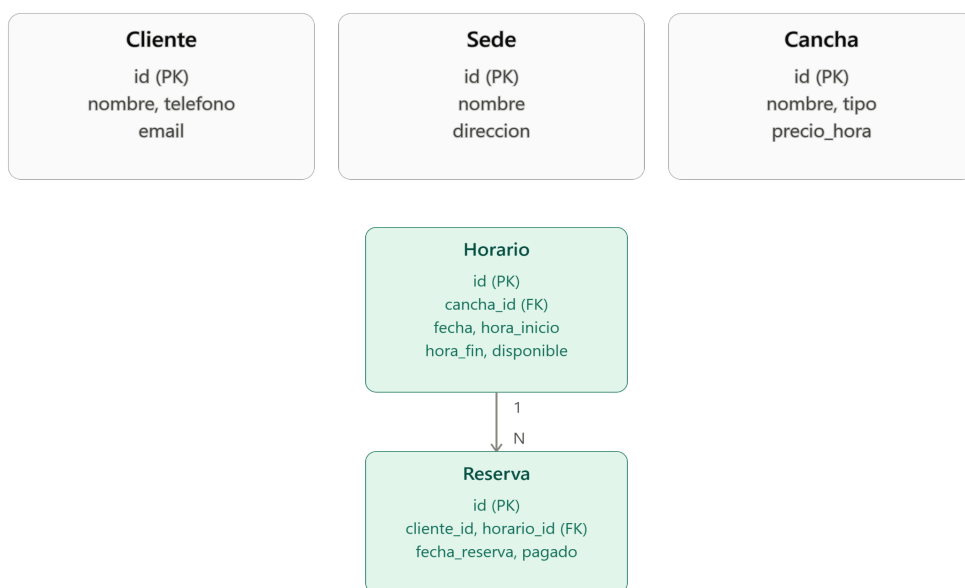
1

|

|

N

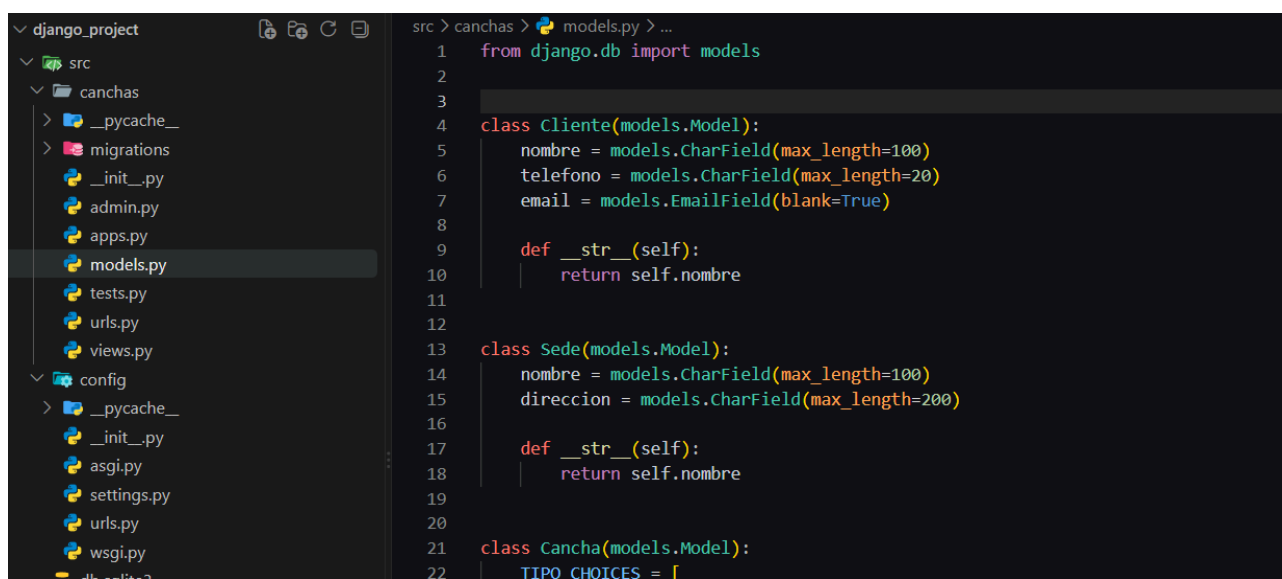
Entidad E



La problemática, los nombres de las entidades y los campos deben ser definidos por el estudiante a partir de su investigación.

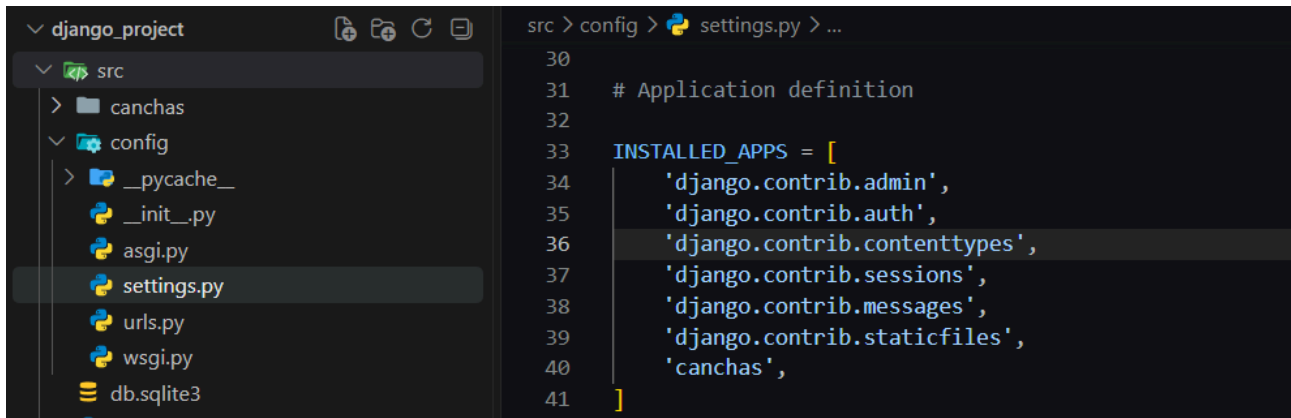
Ejercicio 11 — Crear la aplicación Django

Cree una nueva App Django para implementar la problemática seleccionada. Regístrela correctamente en INSTALLED_APPS y configure sus URLs dentro del Project.



```

src > canchas > models.py > ...
1  from django.db import models
2
3
4  class Cliente(models.Model):
5      nombre = models.CharField(max_length=100)
6      telefono = models.CharField(max_length=20)
7      email = models.EmailField(blank=True)
8
9      def __str__(self):
10         return self.nombre
11
12
13  class Sede(models.Model):
14      nombre = models.CharField(max_length=100)
15      direccion = models.CharField(max_length=200)
16
17      def __str__(self):
18         return self.nombre
19
20
21  class Cancha(models.Model):
22      TIPO_CHOICES = [
  
```

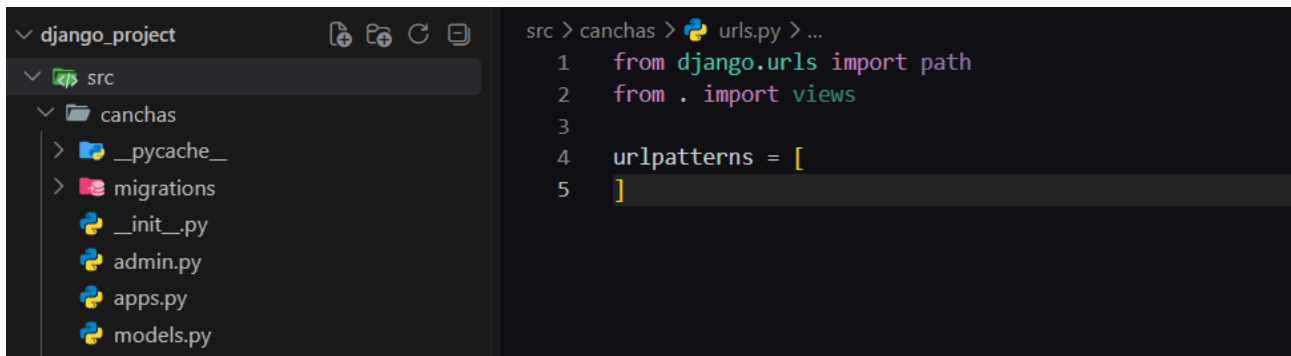


The screenshot shows the Django project structure in the left sidebar. The 'src' folder is expanded, showing 'canchas', 'config', and other files. The 'config' folder is selected, and the 'settings.py' file is open in the editor. The editor shows the 'INSTALLED_APPS' list, which includes 'django.contrib.admin', 'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib.sessions', 'django.contrib.messages', 'django.contrib.staticfiles', and 'canchas'.

```

src > config > settings.py > ...
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'canchas',
41 ]

```



The screenshot shows the Django project structure in the left sidebar. The 'src' folder is expanded, showing 'canchas', 'migrations', and other files. The 'canchas' folder is selected, and the 'urls.py' file is open in the editor. The editor shows the 'urlpatterns' list, which includes 'from django.urls import path' and 'from . import views'.

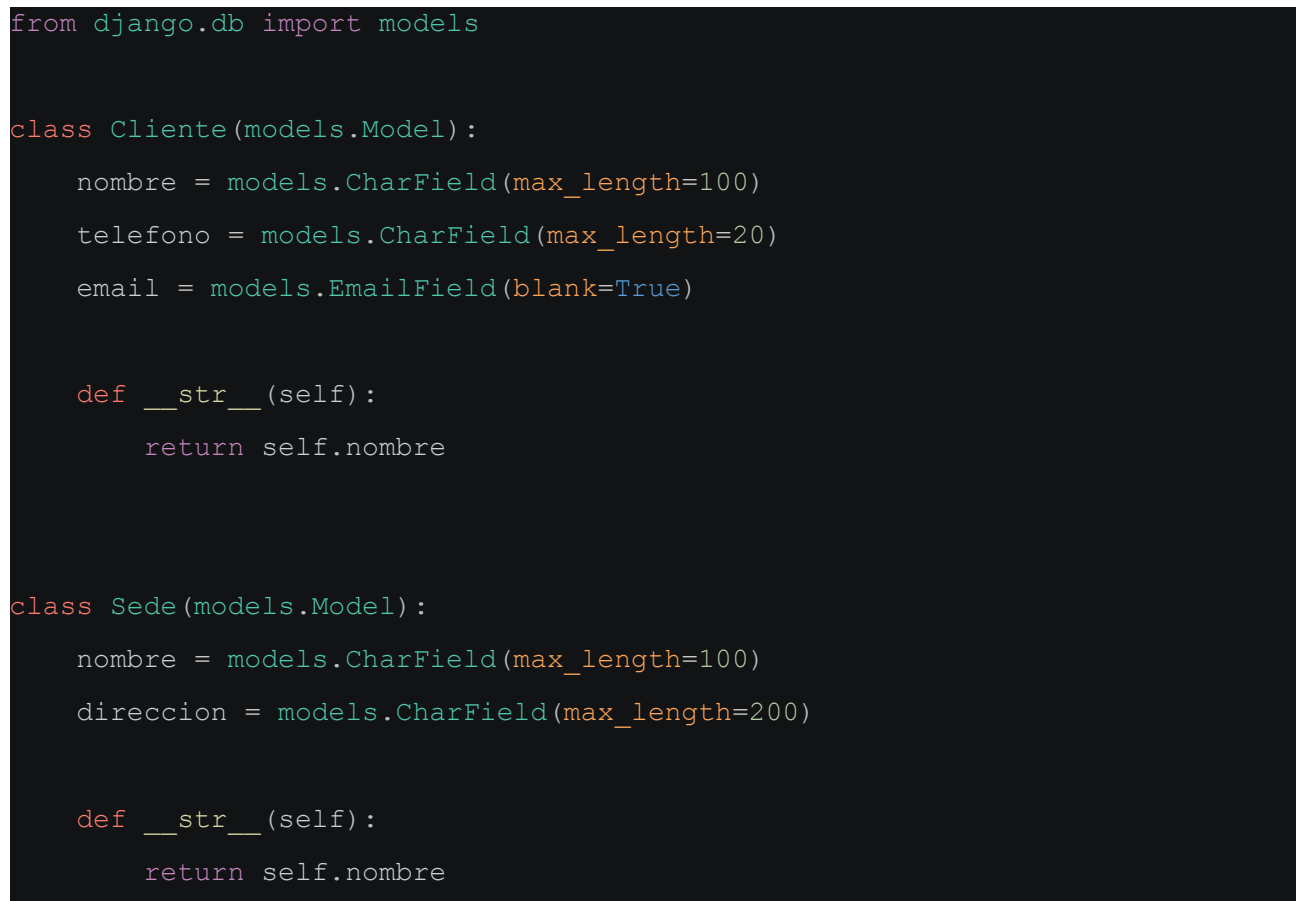
```

src > canchas > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5 ]

```

Ejercicio 12 — Implementar los Models

Traduzca el diseño del Ejercicio 9 a Django Models. Implemente las cinco entidades y configure correctamente la relación ForeignKey entre las dos entidades relacionadas.



The screenshot shows a code editor with the following Django Models code:

```

from django.db import models

class Cliente(models.Model):
    nombre = models.CharField(max_length=100)
    telefono = models.CharField(max_length=20)
    email = models.EmailField(blank=True)

    def __str__(self):
        return self.nombre

class Sede(models.Model):
    nombre = models.CharField(max_length=100)
    direccion = models.CharField(max_length=200)

    def __str__(self):
        return self.nombre

```

```
class Cancha(models.Model):
    TIPO_CHOICES = [
        ('futbol', 'Fútbol'),
        ('voley', 'Vóley'),
        ('tenis', 'Tenis'),
    ]
    nombre = models.CharField(max_length=50)
    tipo = models.CharField(max_length=20, choices=TIPO_CHOICES)
    precio_hora = models.DecimalField(max_digits=8, decimal_places=2)

    def __str__(self):
        return f"{self.nombre} ({self.tipo})"

class Horario(models.Model):
    cancha = models.ForeignKey(Cancha, on_delete=models.CASCADE)
    fecha = models.DateField()
    hora_inicio = models.TimeField()
    hora_fin = models.TimeField()
    disponible = models.BooleanField(default=True)

    def __str__(self):
        return f"{self.cancha} - {self.fecha} {self.hora_inicio}"

class Reserva(models.Model):
    cliente = models.ForeignKey(Cliente, on_delete=models.CASCADE)
    horario = models.ForeignKey(Horario, on_delete=models.CASCADE)
    fecha_reserva = models.DateTimeField(auto_now_add=True)
    pagado = models.BooleanField(default=False)

    def __str__(self):
        return f"Reserva de {self.cliente} - {self.horario}"
```

Ejercicio 13 — Crear la base de datos mediante migraciones

Ejecute las migraciones necesarias para crear la estructura diseñada en SQLite:

```
python manage.py makemigrations
```

```
(venv) PS C:\CARPETA 1\CUARTO_CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\Semana-03\
django_project> venv\Scripts\activate
(venv) PS C:\CARPETA 1\CUARTO_CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\Semana-03\
django_project> cd src
(venv) PS C:\CARPETA 1\CUARTO_CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\Semana-03\
django_project\src> python manage.py makemigrations
Migrations for 'canchas':
  canchas\migrations\0001_initial.py
    + Create model Cancha
    + Create model Cliente
    + Create model Sede
    + Create model Horario
    + Create model Reserva
```

```
python manage.py migrate
```

```
(venv) PS C:\CARPETA 1\CUARTO_CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\Semana-03\
django_project\src> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, canchas, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying canchas.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Verifique posteriormente su estado mediante:

```
python manage.py showmigrations
```

```
(venv) PS C:\CARPETA 1\CUARTO_CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\Semana-03\
django_project\src> python manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
```

```
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
```

canchas

```
[X] 0001_initial
```

contenttypes

```
[X] 0001_initial
```

```
[X] 0002_remove_content_type_name
```

sessions

```
[X] 0001_initial
```

Ejercicio 14 — Implementar CREATE

Implemente una funcionalidad web que permita registrar nuevos datos para las entidades mediante Django ORM. Después de guardar un registro, la aplicación debe redirigir al listado correspondiente.

Conceptualmente:**Formulario**

→ **POST**

→ **View**

→ **ORM**

→ **INSERT**

→ **SQLite**

→ **redirect**

→ **listado**

```
src > canchas > views.py > reserva_create
1  from django.shortcuts import render
2  from django.shortcuts import render, redirect
3  from .models import Cliente, Horario, Reserva
4
5
6  def reserva_create(request):
7      if request.method == 'POST':
8          cliente_id = request.POST['cliente']
9          horario_id = request.POST['horario']
10         Reserva.objects.create(cliente_id=cliente_id, horario_id=horario_id)
11         return redirect('reserva_list')
12
13     clientes = Cliente.objects.all()
14     horarios = Horario.objects.filter(disponible=True)
15     return render(request, 'canchas/reserva_form.html', {
16         'clientes': clientes,
17         'horarios': horarios,
18     })
```

Ejercicio 15 — Implementar READ

Implemente una pantalla de listado para cada entidad utilizando QuerySets. Los datos deberán ser obtenidos desde SQLite mediante Django ORM y mostrados mediante Templates.

Se espera utilizar operaciones como:

Model.objects.all()

Model.objects.filter(...)

Model.objects.order_by(...)

cuando sean necesarias según el problema investigado.

```

24 def cliente_list(request):
25     clientes = Cliente.objects.all().order_by('nombre')
26     return render(request, 'canchas/cliente_list.html', {'clientes': clientes})
27
28 def sede_list(request):
29     sedes = Sede.objects.all().order_by('nombre')
30     return render(request, 'canchas/sede_list.html', {'sedes': sedes})
31
32 def cancha_list(request):
33     canchas = Cancha.objects.all().order_by('tipo')
34     return render(request, 'canchas/cancha_list.html', {'canchas': canchas})
35
36 def horario_list(request):
37     # filter(): solo mostramos los horarios disponibles
38     horarios = Horario.objects.filter(disponible=True).order_by('fecha', 'hora_inicio')
39     return render(request, 'canchas/horario_list.html', {'horarios': horarios})
40
41 def reserva_list(request):
42     reservas = Reserva.objects.all().order_by('-fecha_reserva')
43     return render(request, 'canchas/reserva_list.html', {'reservas': reservas})

```

```

src > canchas > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('clientes/', views.cliente_list, name='cliente_list'),
6      path('sedes/', views.sede_list, name='sede_list'),
7      path('canchas/', views.cancha_list, name='cancha_list'),
8      path('horarios/', views.horario_list, name='horario_list'),
9      path('reservas/', views.reserva_list, name='reserva_list'),
10     path('reservas/nueva/', views.reserva_create, name='reserva_create'),
11 ]

```

Ejercicio 16 — Implementar UPDATE

Cada registro deberá disponer de una opción para actualizar su información. La View deberá recuperar el objeto almacenado, modificar sus propiedades y guardar los cambios mediante Django ORM.

Conceptualmente:

Seleccionar registro

→ GET

→ formulario con datos actuales

→ POST

→ modificar objeto

→ save()

→ ORM

→ UPDATE

→ SQLite

→ redirect

→ listado actualizado

```
37 def cliente_update(request, pk):
38     cliente = get_object_or_404(Cliente, pk=pk)
39     if request.method == 'POST':
40         cliente.nombre = request.POST['nombre']
41         cliente.telefono = request.POST['telefono']
42         cliente.email = request.POST.get('email', '')
43         cliente.save()
44         return redirect('cliente_list')
45     return render(request, 'canchas/cliente_form.html', {'cliente': cliente})
46
```

```
70 def sede_update(request, pk):
71     sede = get_object_or_404(Sede, pk=pk)
72     if request.method == 'POST':
73         sede.nombre = request.POST['nombre']
74         sede.direccion = request.POST['direccion']
75         sede.save()
76         return redirect('sede_list')
77     return render(request, 'canchas/sede_form.html', {'sede': sede})
78
```

```
140 def horario_update(request, pk):
141     horario = get_object_or_404(Horario, pk=pk)
142     if request.method == 'POST':
143         horario.cancha_id = request.POST['cancha']
144         horario.fecha = request.POST['fecha']
145         horario.hora_inicio = request.POST['hora_inicio']
146         horario.hora_fin = request.POST['hora_fin']
147         horario.disponible = 'disponible' in request.POST
148         horario.save()
149         return redirect('horario_list')
150     return render(request, 'canchas/horario_form.html', {
151         'horario': horario,
152         'canchas': Cancha.objects.all().order_by('nombre'),
153     })
154
```

```
140 def horario_update(request, pk):
141     horario = get_object_or_404(Horario, pk=pk)
142     if request.method == 'POST':
143         horario.cancha_id = request.POST['cancha']
144         horario.fecha = request.POST['fecha']
145         horario.hora_inicio = request.POST['hora_inicio']
146         horario.hora_fin = request.POST['hora_fin']
147         horario.disponible = 'disponible' in request.POST
148         horario.save()
149         return redirect('horario_list')
150     return render(request, 'canchas/horario_form.html', {
151         'horario': horario,
152         'canchas': Cancha.objects.all().order_by('nombre'),
153     })
154
```

```
189 def reserva_update(request, pk):
190     reserva = get_object_or_404(Reserva, pk=pk)
191     if request.method == 'POST':
192         reserva.cliente_id = request.POST['cliente']
193         reserva.horario_id = request.POST['horario']
194         reserva.pagado = 'pagado' in request.POST
195         reserva.save()
196         return redirect('reserva_list')
197
198     clientes = Cliente.objects.all()
199     horarios = Horario.objects.all()
200     return render(request, 'canchas/reserva_form.html', {
201         'reserva': reserva,
202         'clientes': clientes,
203         'horarios': horarios,
204     })
205
```

Ejercicio 17 — Implementar DELETE

Cada registro deberá disponer de una opción de eliminación. La aplicación deberá mostrar primero una confirmación y realizar la eliminación mediante una petición POST.

Conceptualmente:

Seleccionar registro

- confirmación
- POST
- delete()
- ORM
- DELETE
- SQLite
- redirect
- listado actualizado

```
48 def cliente_delete(request, pk):
49     cliente = get_object_or_404(Cliente, pk=pk)
50     if request.method == 'POST':
51         cliente.delete()
52         return redirect('cliente_list')
53     return render(request, 'canchas/delete_confirm.html', {
54         'object': cliente,
55         'entity': 'cliente',
56         'cancel_url': 'cliente_list',
57     })
58
59
```

```
114 def cancha_delete(request, pk):
115     cancha = get_object_or_404(Cancha, pk=pk)
116     if request.method == 'POST':
117         cancha.delete()
118         return redirect('cancha_list')
119     return render(request, 'canchas/delete_confirm.html', {
120         'object': cancha,
121         'entity': 'cancha',
122         'cancel_url': 'cancha_list',
123     })
124
```

```
156 def horario_delete(request, pk):
157     horario = get_object_or_404(Horario, pk=pk)
158     if request.method == 'POST':
159         horario.delete()
160         return redirect('horario_list')
161     return render(request, 'canchas/delete_confirm.html', {
162         'object': horario,
163         'entity': 'horario',
164         'cancel_url': 'horario_list',
165     })
166
167
```

```
207 ✓ def reserva_delete(request, pk):
208     reserva = get_object_or_404(Reserva, pk=pk)
209     ✓ if request.method == 'POST':
210         reserva.delete()
211         return redirect('reserva_list')
212     ✓ return render(request, 'canchas/delete_confirm.html', {
213         'object': reserva,
214         'entity': 'reserva',
215         'cancel_url': 'reserva_list',
216     })
```

Canchas	Inicio	Cientes	Sedes	Canchas	Horarios	Reservas
Reservas						
+ Nueva reserva						
Cliente	Horario	Pagado	Acciones			
Juan Diego	sa (voley) - 2026-09-23 11:11:00	True	Editar Eliminar			

Ejercicio 18 — Publicar en GitHub

Actualicen requirements.txt y README.md describiendo su problemática, sus requisitos y la App creada. Hagan commit y push al repositorio del equipo.

<https://github.com/JoelQZ/Lab03DAE>

Entregables

- Cada enunciado debe de tener su respuesta con la justificación y captura de pantalla si lo pide
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente a cada integrante del equipo, según el formato de evidencia indicado en las instrucciones del laboratorio (nombre, título, captura, código, explicación y casos de prueba).

Conclusiones:

- Django ORM permite manejar bases de datos sin escribir SQL directamente. Con los Models se definieron 5 entidades (Cliente, Sede, Cancha, Horario, Reserva) traducidas automáticamente a tablas SQLite mediante migraciones, demostrando la ventaja del ORM para mantener el código más limpio y portable entre motores de base de datos.
- El patrón CRUD (Create, Read, Update, Delete) es la base de cualquier aplicación empresarial con persistencia de datos. Implementar el flujo completo (formulario → POST → View → ORM → SQLite → redirect → listado) permitió entender cómo interactúan los templates, las vistas y la base de datos en Django, además de manejar relaciones ForeignKey para conectar información entre entidades.
- El control de versiones con Git es esencial para documentar el avance y evitar pérdida de trabajo. Se aprendió a usar .gitignore para excluir archivos innecesarios (entorno virtual, caché, base de datos local), y a hacer commits organizados y específicos, evitando mezclar cambios de distintas carpetas o proyectos en un mismo repositorio compartido.